

Web Locks API 深度指南

在浏览器多标签页、多 Worker 并发的世界里，如何安全地协调共享资源？

目录

1. [背景与动机](#)
 2. [核心概念](#)
 3. [基础用法](#)
 4. [锁的模式：独占锁与共享锁](#)
 5. [锁的生命周期与队列机制](#)
 6. [高级特性](#)
 - [ifAvailable：非阻塞尝试](#)
 - [steal：强制抢占](#)
 - [AbortSignal：超时与取消](#)
 1. [查询锁状态](#)
 2. [跨上下文协调](#)
 3. [常见模式与最佳实践](#)
 4. [错误处理](#)
 5. [兼容性与降级策略](#)
 6. [配套案例](#)
-

1. 背景与动机

浏览器里有哪些"并发执行者"？

在深入问题之前，先建立一个直觉：浏览器里不只有"当前页面"在运行代码。

标签页 (Tab) 是最直观的，每个标签页运行一份页面代码。但用户完全可以把同一个网址开两个标签页——比如在一个标签页里看购物车，在另一个标签页里继续逛商品列表。这两个标签页是两个独立的 JavaScript 执行环境，但它们共享同一个浏览器存储 (`localStorage` 、 `IndexedDB`)。

Service Worker 是一段运行在"幕后"的脚本，独立于任何标签页存在。你可以把它想象成一个常驻后台的小程序，即使用户没有打开任何标签页它也可能在运行。它的典型用途是：

- 拦截网络请求，实现离线缓存 (PWA 的核心)
- 接收服务器推送的消息 (比如微信网页版的新消息提醒)
- 在后台同步数据 (比如你在地铁里没网，等到有网时自动把草稿上传)

Web Worker 则是页面主动创建的后台线程，用来把耗时计算 (比如图片压缩、数据加密) 移出主线程，避免卡顿界面。

这三者都可以同时运行，都可以读写 `IndexedDB` 和 `localStorage`。这就埋下了隐患。

场景一：购物车被"覆盖"了

假设你在做一个电商网站，购物车数据存在 `localStorage` 里 (很常见的做法)。

用户同时打开了两个标签页：

- **标签页 A**：用户在商品详情页，点击"加入购物车"
- **标签页 B**：用户在购物车页面，同时删除了一件商品

两个标签页的操作几乎同时发生，代码逻辑都是"读取当前购物车 → 修改 → 写回"：

标签页 A 读取：[商品1, 商品2]

标签页 B 读取：[商品1, 商品2] ← 读到的是同一个旧值

标签页 A 写入：[商品1, 商品2, 商品3] ← 加入了商品3

标签页 B 写入：[商品1] ← 删除了商品2，但基于旧数据写入

最终结果：[商品1]

商品 3 凭空消失了。用户明明点了"加入购物车"，刷新后却不见了。这种 bug 极难复现，因为它只在特定的时序下才会出现。

场景二：离线笔记被"推送"覆盖

你在用一个支持离线的笔记应用（比如 Notion 的简化版）。

你在地铁里没有网络，编辑了一篇笔记，写了很多内容。地铁出站，手机重新联网。此时：

- **你的标签页**：检测到网络恢复，开始把本地修改上传到服务器
- **Service Worker**（后台小程序）：同时收到服务器的推送通知，说"有新版本的笔记，请同步"，于是也开始把服务器的版本写入本地 `IndexedDB`

两者的操作序列如下：

标签页：读取本地笔记（你写的内容）→ 上传中...

Service Worker：收到推送 → 读取服务器笔记（旧版本）→ 写入本地

标签页：上传完成，更新本地时间戳

如果 Service Worker 的写入发生在标签页读取之后、上传完成之前，你本地的笔记就被服务器的旧版本覆盖了。你辛苦写的内容消失了。

这个 bug 的可怕之处在于：**Service Worker 是完全独立运行的**，它不知道你的标签页正在做什么，标签页也不知道 Service Worker 在做什么。它们之间没有任何天然的协调机制。

场景三：重复提交订单

用户在结账页面点击"提交订单"按钮。网络稍慢，用户以为没反应，又点了一次。或者更常见的情况：用户同时在两个标签页里都进入了结账流程。

前端通常用一个变量来防重复：

```
let isSubmitting = false;

async function submitOrder() {
  if (isSubmitting) return; // 防重复
  isSubmitting = true;

  await fetch('/api/orders', { method: 'POST', body: orderData });

  isSubmitting = false;
}
```

这个方案在单标签页里完全有效。但如果用户在两个标签页里同时点击提交，

`isSubmitting` 是各自标签页的局部变量，互相不可见。两个标签页都会认为"当前没有在提交"，于是都发出了请求，产生了重复订单。

场景四：后台数据同步的"踩踏"

一个 PWA（可以安装到桌面的网页应用，比如 Twitter Lite）通常有这样的逻辑：定期把本地数据和服务器同步。

问题是，这个同步逻辑可能在多个地方被触发：

- 用户打开应用时（标签页 A）
- 用户在另一个标签页里也打开了应用（标签页 B）
- Service Worker 的定时后台同步也触发了

三个同步任务同时运行，都在读取本地数据库、请求服务器、写回结果。轻则浪费带宽（发了三次重复请求），重则数据写入顺序混乱，导致本地数据库状态不一致。

为什么这些问题在 Web 上特别难处理？

在服务端，这类并发问题有成熟的解决方案：数据库锁、Redis 分布式锁、消息队列.....但在浏览器端，长期以来没有原生的互斥机制。

开发者只能用一些"土方法"来应对：

**** 方法一：用 `localStorage` 设置标志位 ****

```
// 标签页 A
localStorage.setItem('syncing', 'true');
await doSync();
localStorage.removeItem('syncing');

// 标签页 B
if (localStorage.getItem('syncing') === 'true') return; // 跳过
```

问题：`setItem` 和后续的检查不是原子操作。两个标签页可能同时读到 `null`，同时认为"没有在同步"，于是都设置了标志位，都开始同步。而且 `localStorage` 在 Worker 里根本无法访问。

方法二：用 ** BroadcastChannel 协商 ******

`BroadcastChannel` 允许同源的标签页和 Worker 互相发消息。可以用它来实现一个"谁先喊谁做"的协议：

```
const bc = new BroadcastChannel('sync-channel');

bc.postMessage({ type: 'want-to-sync', tabId: myId });

bc.onmessage = (e) => {
  if (e.data.type === 'sync-granted' && e.data.to === myId) {
    doSync();
  }
};
```

问题：你需要自己实现完整的消息握手协议——谁是"仲裁者"？仲裁者崩溃了怎么办？消息丢失了怎么办？超时了怎么办？这套逻辑写起来比你的业务代码还复杂，而且极难测试。

方法三：依赖 IndexedDB 的事务

IndexedDB 的事务本身是有隔离性的，可以用它来保护数据库操作的原子性。

```
const tx = db.transaction('notes', 'readwrite');
const note = await tx.objectStore('notes').get(id);
note.content = newContent;
await tx.objectStore('notes').put(note);
```

问题：这只能保护 IndexedDB 内部的操作。如果你的业务逻辑还涉及网络请求（先读数据库，再发请求，再写数据库），IndexedDB 事务无法跨越这个异步边界——事务会在你 `await fetch()` 的时候自动提交并关闭。

Web Locks API 的出现

2019 年，Web Locks API 进入 Chrome 69，随后被 Firefox 96、Safari 15.4 跟进。它提供了一个**操作系统级互斥锁**的浏览器原生抽象：

- **跨上下文**：标签页、Worker、Service Worker 共享同一锁空间
- **自动释放**：回调结束或上下文销毁时锁自动释放，不会死锁
- **声明式**：无需手动实现队列，浏览器负责排队和调度
- **可组合**：支持共享锁、独占锁、超时、强制抢占等模式

回到场景一的购物车问题，有了 Web Locks，解法变得非常直接：

```
async function updateCart(operation) {
  await navigator.locks.request('shopping-cart', async () => {
    // 在锁的保护下，读取-修改-写入是原子的
    // 无论有多少标签页同时调用这个函数，都会排队依次执行
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    operation(cart);
    localStorage.setItem('cart', JSON.stringify(cart));
  });
}
```

2. 核心概念

锁 (Lock)

Web Locks API 中的"锁"是一个**命名资源**的访问令牌。锁名称是任意字符串，代表你想要保护的逻辑资源（不必与实际文件或数据库对应）。

```
lock name: "user-profile-42"    ← 逻辑资源名，自定义
lock mode: "exclusive"         ← 访问模式
```

锁空间 (Lock Scope)

同一源 (origin) 下的所有浏览上下文共享同一个锁空间。`https://app.example.com` 下的所有标签页和 Worker 使用同一套锁，与 `https://other.example.com` 完全隔离。

请求与持有

```
请求锁 → [等待队列] → 获得锁 → 执行回调 → 回调结束 → 释放锁
```

锁的持有时间 = 回调函数的执行时间。回调返回的 Promise resolve 或 reject 时，锁立即释放。

3. 基础用法

API 签名

```
navigator.locks.request(name, [options], callback)
```

- `name`：锁的名称（字符串）
- `options`：可选配置项
- `callback`：获得锁后执行的函数，**必须返回 Promise**
- 返回值：一个 Promise，resolve 值为 callback 的返回值

最简示例

```
// 获取名为 "my-resource" 的独占锁
const result = await navigator.locks.request('my-resource', async (lock) => {
  // 此时我们独占持有该锁
  console.log('持有锁: ', lock.name, lock.mode);

  await doSomethingCritical();

  return '操作完成'; // 这个值会成为 request() 的 resolve 值
});

console.log(result); // '操作完成'
// 锁已自动释放
```

为什么回调必须返回 Promise?

锁的持有时间由 Promise 的生命周期决定。如果回调是同步的，锁会在回调返回后立即释放——这通常不是你想要的。


```
// ❌ 错误：锁在 fetch 发出后就释放了
navigator.locks.request('data', (lock) => {
  fetch('/api/data').then(save); // 没有 await, 没有返回 Promise
});

// ✅ 正确：锁持续到 fetch 和 save 都完成
navigator.locks.request('data', async (lock) => {
  const data = await fetch('/api/data');
  await save(data);
});
```

4. 锁的模式：独占锁与共享锁

`mode` 只有两个值：`'exclusive'`（默认）和 `'shared'`。它们描述的是同一个锁名称上的两种请求方式，浏览器根据当前持有情况决定新请求是立即获得还是排队等待。

独占锁（exclusive）

同一时刻只有一个持有者，其他任何请求（无论 `shared` 还是 `exclusive`）都必须等待。不指定 `mode` 时默认就是独占锁。

```
await navigator.locks.request('my-resource', async () => {
  // 此时没有任何其他代码能同时持有 'my-resource'
  const value = await readCounter();
  await writeCounter(value + 1);
});
```

共享锁（shared）

多个 `shared` 请求可以同时持有同一个锁，互不阻塞。但只要有任何一个 `exclusive` 请求存在（无论是正在持有还是在等待），`shared` 就必须等待。

```
// 这两个调用会同时运行，互不等待
navigator.locks.request('my-resource', { mode: 'shared' }, async () => {
  console.log('读者 A 开始');
  await sleep(1000);
  console.log('读者 A 结束');
});

navigator.locks.request('my-resource', { mode: 'shared' }, async () => {
  console.log('读者 B 开始'); // 和读者 A 同时打印
  await sleep(1000);
  console.log('读者 B 结束');
});
```

两种模式针对的是同一个锁名

这是最容易混淆的地方：**shared 和 exclusive 不是两把不同的锁，而是对同一个锁名的两种请求方式**。

```
// 以下三个请求，操作的都是同一个叫 'my-resource' 的锁
navigator.locks.request('my-resource', { mode: 'shared' }, callback); //
请求 A
navigator.locks.request('my-resource', { mode: 'shared' }, callback); //
请求 B
navigator.locks.request('my-resource', { mode: 'exclusive' }, callback); //
请求 C
```

浏览器的调度规则是：

新请求 \ 当前持有	无锁	shared	exclusive
shared 请求	✅ 立即获得	✅ 立即获得	🕒 等待
exclusive 请求	✅ 立即获得	🕒 等待	🕒 等待

所以上面三个请求的执行顺序是：A 和 B 同时获得锁并发执行，C 等 A 和 B 都完成后才能获得锁。

这个设计有什么用？

`shared` 模式存在的意义是：允许"只读"操作并发，同时保证"写"操作的独占性。

如果你的业务里，读操作远多于写操作，而且多个读操作同时进行不会有问题（读不改变数据），那么就可以利用这个特性提升并发性能：

```
// 约定：读操作用 shared，写操作用 exclusive，请求同一个锁名
async function readNote(id) {
  return navigator.locks.request(`note-${id}`, { mode: 'shared' }, async ()
=> {
    return await db.get(id); // 多个读者可以同时进行
  });
}

async function writeNote(id, content) {
  return navigator.locks.request(`note-${id}`, { mode: 'exclusive' }, async
() => {
    await db.put(id, content); // 写入时独占，等所有读者完成后才执行
  });
}
```

这种"读并发、写独占"的模式在并发理论里叫做读写锁（Readers–Writer Lock），但它不是一种新的 API——只是你在代码里做了一个约定：读用 `shared`，写用 `exclusive`，锁名相同。Web Locks 的调度规则自然就实现了这个效果。

如果你不需要区分读写，直接用默认的 `exclusive` 即可，简单可靠。`shared` 模式只在你明确需要允许并发读取时才有价值。

5. 锁的生命周期与队列机制

请求排队

当锁被持有时，新的请求会进入**等待队列**，按 FIFO 顺序依次获得锁：

时间轴：

t=0 A 请求独占锁 → 立即获得
t=1 B 请求独占锁 → 进入队列等待
t=2 C 请求独占锁 → 进入队列等待
t=3 A 的回调完成 → 释放锁 → B 获得锁
t=5 B 的回调完成 → 释放锁 → C 获得锁

锁的自动释放

以下情况会自动释放锁，**无需手动操作**：

1. 回调函数返回的 Promise **resolve**
2. 回调函数返回的 Promise **reject**（锁释放，错误向上传播）
3. 持有锁的**浏览上下文被销毁**（标签页关闭、Worker 终止）

这是 Web Locks API 相比手动实现的最大优势——**不会死锁**。

锁的嵌套

同一上下文可以请求自己已持有的锁，但要注意**死锁风险**：

```
// ✅ 安全：不同名称的锁可以嵌套
await navigator.locks.request('lock-A', async () => {
  await navigator.locks.request('lock-B', async () => {
    // 持有 A 和 B
  });
});

// ⚠️ 死锁！同一名称的独占锁不能嵌套
await navigator.locks.request('lock-A', async () => {
  // 这里会永远等待，因为 'lock-A' 已被当前上下文持有
  await navigator.locks.request('lock-A', async () => { /* 永远不会执行 */
});
});
```

6. 高级特性

6.1 ifAvailable: 非阻塞尝试

`ifAvailable: true` 时，如果锁当前不可用，立即以 **`null`** 调用回调，而不是等待。

```
await navigator.locks.request('resource', { ifAvailable: true }, async
(lock) => {
  if (lock === null) {
    // 锁不可用，跳过或稍后重试
    console.log('资源正忙，跳过本次同步');
    return;
  }

  // 成功获得锁
  await syncData();
});
```

适用场景：

场景一：后台定期同步任务（忙则跳过）

假设你的应用每 30 秒自动把本地数据同步到服务器。如果上一次同步还没完成，这次就跳过，等下次再试——重复同步没有意义，还会浪费带宽。

```
setInterval(async () => {
  await navigator.locks.request('background-sync', { ifAvailable: true },
    async (lock) => {
      if (lock === null) {
        console.log('上次同步还在进行，跳过');
        return; // 直接返回，不等待
      }
      await syncToServer();
    });
}, 30_000);
```

如果用普通的 `request()`（不加 `ifAvailable`），第二次触发时会排队等待第一次完成，然后立刻再跑一次同步——完全没有意义。

场景二：非关键性的乐观更新

"乐观更新"是指：用户点击点赞按钮，界面立刻显示已点赞，同时在后台发请求给服务器。这种操作不需要精确串行——如果当前已经有一个点赞请求在处理，新的点击直接跳过即可（或者用最新状态覆盖），不需要排队。

```
async function toggleLike(postId) {
  await navigator.locks.request(`like-${postId}`, { ifAvailable: true },
    async (lock) => {
      if (lock === null) {
        // 上一次点击还在处理中，忽略这次重复点击
        return;
      }
      await fetch(`/api/posts/${postId}/like`, { method: 'POST' });
    });
}
```

如果用普通锁，用户快速双击点赞按钮会发出两次请求，可能导致先点赞后取消赞（或反过来），结果取决于哪个请求先到达服务器——这是竞态。用 `ifAvailable` 直接丢弃第二次点击，更符合预期。

场景三：避免长时间阻塞 UI

有些操作（比如导出大文件、批量处理）持有锁的时间很长。如果用户触发了另一个需要同一把锁的操作，与其让用户界面一直转圈等待，不如立刻告知“当前有任务在进行，请稍后再试”。

```
async function exportData() {
  await navigator.locks.request('export', { ifAvailable: true }, async
(lock) => {
    if (lock === null) {
      alert('导出任务正在进行中，请等待完成后再试');
      return;
    }
    await generateAndDownloadFile(); // 可能需要几秒钟
  });
}
```

这比让用户盯着转圈等待（且不知道要等多久）体验好得多。

6.2 steal：强制抢占

`steal: true` 的行为可以用一句话描述：无视一切队列，立刻拿到锁，同时把所有相关方都踢开。

具体来说，当你发出一个 `steal` 请求时，浏览器会做三件事：

1. 中止所有正在持有该锁的请求——它们的回调会立刻收到 `AbortError`，Promise 以错误结束
2. 清空等待队列——所有正在排队等待该锁的请求，同样收到 `AbortError`，全部作废
3. 立刻把锁交给你——不管之前有多少人在持有或等待

```
// 紧急情况：强制获取锁
await navigator.locks.request('critical-resource', { steal: true }, async
(lock) => {
  await emergencyCleanup();
});
```

用一个具体的例子来理解这三种情况：

假设当前锁的状态是：

持有中：请求 A（正在执行写入数据库）
等待中：请求 B（排队中）
等待中：请求 C（排队中）

此时请求 D 带着 `steal: true` 进来，结果是：

请求 A：回调立刻收到 `AbortError`，写入操作被中断（数据库可能处于中间状态！）
请求 B：收到 `AbortError`，永远不会执行
请求 C：收到 `AbortError`，永远不会执行
请求 D：立刻获得锁，开始执行

**** A、B、C 的代码需要自己捕获这个错误： ****

```
// 请求 A 的代码（被 steal 中断）
try {
  await navigator.locks.request('critical-resource', async () => {
    await db.write(step1);
    await db.write(step2); // ← 如果 steal 发生在这里，step2 不会执行
  });
} catch (err) {
  if (err.name === 'AbortError') {
    // 我被强制中断了，需要做清理或回滚
    await db.rollback();
  }
}
```



```
}
```

**** `steal` ** 对等待队列的影响**

等待队列里的请求（B 和 C）会被 **全部清空**，而不是让 D 插队到队列头部。这意味着 D 执行完之后，B 和 C 不会自动恢复——它们已经以 `AbortError` 结束了，如果还需要执行，调用方必须自己重试。

什么时候才应该用 ** `steal` ? **

`steal` 的破坏性很大，只有在“不得不强行恢复控制权”的极端场景下才应该使用：

- ** 页面卸载前的紧急清理 **：用户关闭标签页，`beforeunload` 事件触发，你需要立刻把某个标志位写入存储，但锁正被一个长时间操作持有，等不了

```
window.addEventListener('beforeunload', () => {  
  // 同步的 beforeunload 里无法 await，但可以发起 steal 请求  
  navigator.locks.request('session', { steal: true }, async () => {  
    await markSessionAsEnded();  
  });  
});
```

- 检测到锁被“僵尸进程”持有：某个 Worker 崩溃后，理论上锁会自动释放，但在极少数异常情况下你需要强制恢复

⚠ 谨慎使用： `steal` 会把正在进行的操作强行中断，被中断的代码如果没有妥善处理 `AbortError`，很容易留下不一致的数据状态。在绝大多数业务场景里，你都不需要它——老老实实排队等待是更安全的选择。

`steal` ** 与 ** `ifAvailable` ** 不能同时使用**，会抛出 `NotSupportedError`。

6.3 AbortSignal：超时与取消

通过 ** `AbortController` ** 可以取消等待中**的锁请求（不能取消已持有的锁）。

```
// 5 秒超时
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);

try {
  await navigator.locks.request(
    'slow-resource',
    { signal: controller.signal },
    async (lock) => {
      await doWork();
    }
  );
} catch (err) {
  if (err.name === 'AbortError') {
    console.log('等待锁超时，操作已取消');
  }
} finally {
  clearTimeout(timeoutId);
}
```

** 使用 `AbortSignal.timeout()` (更简洁): **

```
try {
  await navigator.locks.request(
    'resource',
    { signal: AbortSignal.timeout(5000) },
    async (lock) => {
      await doWork();
    }
  );
} catch (err) {
  if (err.name === 'TimeoutError') {
    console.log('超时');
  }
}
```

重要限制：AbortSignal 只能取消等待，不能中断执行

这是最容易误解的地方。**signal** 只在锁请求**还在等待队列中**时有效。一旦回调开始执行（锁已持有），即使你调用了 `controller.abort()`，回调也不会被中断，它会继续跑到结束。

时间轴：

t=0 发出锁请求，进入等待队列 ← `abort()` 在这里有效，会取消等待
t=2 获得锁，回调开始执行 ← 从这一刻起，`abort()` 对锁本身无效
t=5 回调执行完毕，锁释放

如果你需要在回调执行过程中也能响应取消信号，必须在回调内部**手动检查**

`signal.aborted`：

```
const controller = new AbortController();
const { signal } = controller;

await navigator.locks.request('resource', { signal }, async () => {
  // 锁已持有，signal.abort() 不会中断这里的代码
  // 需要自己在关键节点检查

  await step1();

  if (signal.aborted) {
    await rollback(); // 手动回滚
    return;
  }

  await step2();

  if (signal.aborted) {
    await rollback();
    return;
  }

  await step3();
});
```

这个设计是有意为之的：锁一旦持有，持有者就对资源有完整控制权，外部不能强行打断（那是 `steal` 的职责，且代价更大）。`signal` 的作用仅限于“我不想再等了，放弃排队”。

7. 查询锁状态

`navigator.locks.query()` 返回当前锁空间的快照，用于调试和监控：

```
const state = await navigator.locks.query();

console.log('持有中的锁: ', state.held);
// [{ name: 'user-data', mode: 'exclusive', clientId: '...' }]

console.log('等待中的锁: ', state.pending);
// [{ name: 'user-data', mode: 'exclusive', clientId: '...' }]
```

每个锁条目包含：

- `name`：锁名称
- `mode`：'exclusive' 或 'shared'
- `clientId`：持有/等待该锁的浏览上下文 ID

**** 关于 `clientId` ****

"浏览上下文"是浏览器对"一个独立 JS 运行环境"的统一称呼。每一个标签页、iframe、Service Worker、Web Worker 在创建时都会被分配一个唯一的 UUID，这就是 `clientId`。

它的主要用途是调试——当你看到某把锁被持有时，可以通过 `clientId` 判断是哪个上下文在持有它：

```
const state = await navigator.locks.query();

for (const lock of state.held) {
  console.log(lock.clientId);
  // 输出类似: '7f3a2b1c-4d5e-6f7a-8b9c-0d1e2f3a4b5c'
}
```

在日常业务代码里几乎用不到 `clientId`，它的价值是**对比**：判断某把锁是不是被当前上下文自己持有，或者在调试“锁被哪个标签页卡住了”时作为参考。

⚠️ `query()` 返回的是**时间点快照**，结果可能在你读取时就已过期。不要用它来做锁决策，仅用于调试。

8. 跨上下文协调

Web Locks API 最强大的场景是跨标签页和 Worker 的协调。同一个源下的所有上下文共享同一个锁空间，这意味着一个标签页请求的锁，另一个标签页或 Service Worker 完全感知得到。

场景一：防止重复同步

问题： 用户同时打开了应用的三个标签页，每个标签页在启动时都会触发一次“从服务器拉取最新数据”的操作。结果三个标签页同时发出了三次完全相同的请求，浪费带宽，还可能因为写入顺序不确定导致数据混乱。

解法： 用锁保证同一时刻只有一个标签页在执行同步，其他标签页发现“已有人在同步”就跳过，等同步完成后直接读取已更新的本地数据即可。

```
// 三个标签页都运行这段代码，但只有一个能同时执行
async function syncWithServer() {
  await navigator.locks.request('server-sync', { ifAvailable: true }, async
(lock) => {
    if (lock === null) {
      // 已有其他标签页在同步，等它完成后我直接读本地数据就好
      return;
    }

    const lastSync = await db.get('lastSync');
    if (Date.now() - lastSync < 60_000) return; // 1 分钟内已同步，跳过

    const data = await fetch('/api/data').then(r => r.json());
    await db.put('data', data);
    await db.put('lastSync', Date.now());
  });
}
```

场景二：主标签页选举

问题： 有些任务只应该由一个标签页来做，比如维持 WebSocket 长连接、定时轮询服务器、处理推送消息分发。如果每个标签页都各自维护一个 WebSocket，既浪费连接资源，收到消息时也会重复处理。

解法： 利用 `ifAvailable` 做选举——第一个成功拿到锁的标签页成为"主标签页"，负责维护 WebSocket；其他标签页成为"从标签页"，通过 `BroadcastChannel` 接收主标签页转发的消息。主标签页关闭时，锁自动释放，其他标签页重新竞争，产生新的主标签页。

```

async function electLeader() {
  await navigator.locks.request(
    'leader-lock',
    { ifAvailable: true },
    async (lock) => {
      if (lock === null) {
        // 已有主标签页，我作为从标签页监听广播
        listenForBroadcast();
        return;
        // 注意：这里直接 return，锁不会被持有
        // 当主标签页关闭时，需要重新触发选举
      }

      // 我成为主标签页，持有锁直到我自己关闭
      // 锁会在标签页关闭时自动释放，触发其他标签页重新选举
      console.log('我是主标签页，建立 WebSocket 连接');
      const ws = new WebSocket('wss://api.example.com/live');

      ws.onmessage = (e) => {
        // 把消息广播给所有从标签页
        new BroadcastChannel('ws-messages').postMessage(e.data);
      };

      // 持有锁直到标签页关闭
      await new Promise(() => {}); // 永不 resolve，锁一直持有
    }
  );
}

function listenForBroadcast() {
  const bc = new BroadcastChannel('ws-messages');
  bc.onmessage = (e) => handleMessage(e.data);

  // 监听主标签页是否下线（锁释放后重新选举）
  navigator.locks.request('leader-lock', async () => {
    // 能走到这里说明主标签页已关闭，我来竞选
    await electLeader();
  });
}

```

场景三：Service Worker 与标签页的写入协调

问题： 你的应用支持离线，本地数据存在 IndexedDB 里。用户在标签页里编辑数据，同时 Service Worker 收到服务器推送，也要往同一个 IndexedDB 写入新数据。两者同时写入，谁后写谁覆盖谁，用户的修改可能丢失。

解法： 标签页和 Service Worker 约定使用同一把锁来保护写入操作。因为它们共享同一个锁空间，这个约定天然有效。

```
// 标签页里的保存操作
async function saveNote(id, content) {
  await navigator.locks.request(`note-write-${id}`, async () => {
    await db.put('notes', { id, content, updatedAt: Date.now() });
  });
}

// service-worker.js 里的推送处理
self.addEventListener('push', (event) => {
  event.waitUntil(async function () {
    const { id, content } = event.data.json();

    // 和标签页请求同一把锁，自动排队，不会同时写入
    await navigator.locks.request(`note-write-${id}`, async () => {
      const local = await db.get('notes', id);
      // 只有服务器版本更新时才覆盖
      if (!local || local.updatedAt < serverUpdatedAt) {
        await db.put('notes', { id, content });
      }
    });
  })();
});
```

场景四：Web Worker 的计算结果回写

问题： 主线程把一个大任务（比如解析一个大 CSV 文件）交给 Web Worker 处理。Worker 处理完后需要把结果写入 IndexedDB。但主线程可能同时也在写入同一个表（比如用户手动输入了一条记录）。

解法：Worker 和主线程都通过同一把锁来保护写入，无论谁先完成，都能安全地串行写入。

```
// 主线程
async function saveUserInput(data) {
  await navigator.locks.request('db-write', async () => {
    await db.put('records', data);
  });
}

// worker.js
self.onmessage = async (e) => {
  const result = await parseCSV(e.data.file); // 耗时处理

  // Worker 里同样可以使用 Web Locks
  await navigator.locks.request('db-write', async () => {
    for (const row of result) {
      await db.put('records', row);
    }
  });

  self.postMessage({ done: true });
};
```

9. 常见模式与最佳实践

模式一：防重复提交

```
let isSubmitting = false; // ❌ 仅在单标签页有效

// ✅ 跨标签页防重复
async function submitForm(data) {
  await navigator.locks.request('form-submit', { ifAvailable: true }, async
(lock) => {
    if (lock === null) {
      showToast('正在提交中，请勿重复操作');
      return;
    }

    await fetch('/api/submit', { method: 'POST', body: JSON.stringify(data)
});
  });
}
```

模式二：原子性读-改-写

```
async function incrementCounter(key) {
  return navigator.locks.request(`counter:${key}`, async () => {
    const current = await idb.get(key) ?? 0;
    await idb.put(key, current + 1);
    return current + 1;
  });
}
```

模式三：带超时的锁请求

```
async function withTimeout(lockName, timeout, callback) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeout);

  try {
    return await navigator.locks.request(
      lockName,
      { signal: controller.signal },
      callback
    );
  } catch (err) {
    if (err.name === 'AbortError') {
      throw new Error(`获取锁 "${lockName}" 超时 (${timeout}ms)`);
    }
    throw err;
  } finally {
    clearTimeout(timer);
  }
}

// 使用
await withTimeout('slow-resource', 3000, async (lock) => {
  await doWork();
});
```

模式四：锁包装器（装饰器模式）

```
function withLock(lockName, options = {}) {
  return function decorator(fn) {
    return async function (...args) {
      return navigator.locks.request(lockName, options, () =>
fn.apply(this, args));
    };
  };
}

// 使用
const syncData = withLock('data-sync')(async function () {
  await fetch('/api/sync');
});
```

最佳实践总结

原则	说明
最小化锁持有时间	只在真正需要互斥的代码段持有锁，避免在锁内做不必要的 I/O
细粒度命名	用 <code>resource:id</code> 格式命名，避免过于宽泛的锁名导致不必要的串行化
始终处理 <code>AbortError</code>	使用 <code>signal</code> 时必须捕获取消错误
不要依赖 <code>query()</code> 做决策	<code>query()</code> 仅用于调试，结果是过时快照
避免锁内嵌套同名锁	会导致死锁
回调必须返回 <code>Promise</code>	否则锁会提前释放

10. 错误处理

```
try {
  await navigator.locks.request('resource', { signal }, async (lock) => {
    await riskyOperation();
  });
} catch (err) {
  switch (err.name) {
    case 'AbortError':
      // signal 被中止 (超时或手动取消)
      console.log('锁请求被取消');
      break;
    case 'NotSupportedError':
      // 非法选项组合 (如 steal + ifAvailable)
      console.error('无效的锁选项');
      break;
    case 'SecurityError':
      // 在不安全上下文 (非 HTTPS) 中使用
      console.error('需要安全上下文 (HTTPS) ');
      break;
    default:
      // 回调内部抛出的错误也会传播到这里
      console.error('操作失败: ', err);
  }
}
```

11. 兼容性与降级策略

浏览器支持（截至 2026 年）

浏览器	支持版本
Chrome	69+
Edge	79+
Firefox	96+
Safari	15.4+
Chrome Android	69+
Safari iOS	15.4+

全球覆盖率约 96%+，主流应用可直接使用。

特性检测

```
if ('locks' in navigator) {
  // 使用 Web Locks API
  await navigator.locks.request('resource', callback);
} else {
  // 降级：直接执行（接受竞态风险）或使用 polyfill
  await callback({ name: 'resource', mode: 'exclusive' });
}
```

Polyfill

对于需要支持旧浏览器的场景，可使用 `web-locks-polyfill`，基于 IndexedDB 实现跨标签页锁。

12. 配套案例

本目录包含以下 HTML 演示文件，可直接在浏览器中打开运行：

文件	内容
01-basic-lock.html	基础锁获取与释放，观察锁的生命周期
02-shared-exclusive.html	共享锁与独占锁的并发行为对比
03-counter-race.html	竞态条件演示：有锁 vs 无锁的计数器
04-lock-queue.html	锁队列可视化，实时观察等待与持有状态
05-abort-signal.html	AbortSignal 超时取消锁请求
06-cross-tab-sync.html	跨标签页同步：主标签页选举与互斥写入

参考资料

- [MDN Web Docs: Web Locks API](#)
- [W3C Web Locks API 规范](#)
- [web.dev: Web Locks API](#)
- [Chrome Platform Status](#)